T. P.
Arbres de régression, classification

Résumé

Ce TP reprend rapidement des éléments du cours et propose une mise en pratique interactive des arbres de décision. La technique de l'arbre de décision est employée en classement pour détecter des critères permettant de répartir les individus d'une population en n classes. On commence par choisir la variable qui sépare le mieux les individus en plusieurs classes (généralement 2). Nous avons alors des sous populations distinctes, à partir d'un ensemble initial, que l'on appellera nœud. On réitère la même opération sur chaque nouveau nœud obtenu jusqu'à ce que la séparation des individus ne soit plus pertinente (nous définirons cette notion de pertinence plus tard). Les nœuds finaux sont appelés *feuilles*. Après la construction de l'arbre nous obtenons des feuilles caractérisant une seule classe. Un individu est affecté à une feuille (sous entendu : une classe), avec une certaine probabilité, lorsqu'il satisfait l'ensemble des règles lui permettant d'aboutir à la feuille. L'ensemble des règles de toutes les feuilles constitue le modèle de classement.

Les arbres de décision conçus pour le classement permettent généralement d'effectuer une prédiction, en faisant varier le critère de scission pour chaque nœuds (à l'exception de quelques techniques particulières, comme QUEST, qui combine toutes les variables pour choisir le critère de scission). Dans un arbre de régression, on travaille exclusivement sur des variables quantitatives continues. On cherche à prédire la valeur d'une variable pré-définie en entrée de la fonction, puis on construit un arbre à partir des autres variables explicatives. Le principe de l'arbre est le suivant :

- La variable à expliquer doit avoir une variance plus faible dans les nœds-fils que le nœud-père [1].
- La variable à expliquer doit avoir une moyenne la plus distincte possible d'un nœud-fils à un autre.

Il faut donc choisir des nœuds-fils qui minimisent la variance intraclasse et maximise la variance interclasse. On peut appliquer un test de Fisher-Snedecor au rapport *Variance intraclasse / Variance interclasse*.

On considère qu'il faut séparer un nœud lorsque le rapport des variances a au plus 20% de chance d'être aussi élevé si la variable à expliquer est indépendante de la variables choisie pour scinder le nœud-père. Intuitivement on sépare un nœud lorsqu'on a au moins 20% de chance d'observer une différence sur la variance (entre le nœud initial et le nœud séparé).

1. Au niveau le plus élevé il y a un nœud racine, dit : nœud père. Au niveau directement inférieur, il y a au moins deux nœuds fils

1 Arbre de régression

1.1 Chargement des données

On décide d'observer les différences de poids pour une population de nouveaux nés dans un service de recherche.
Nous avons comme information :

- X : l'ID du bébé évalué
- *bwt* : poids à la naissance (en onces, soit 1 once = soit 28,349 g)
- *gestation* : durée de la grossesse (en jours)
- *parity* : 1 si le bébé est né en premier, 0 sinon
- *age* : âge de la mère (en années)
- *height* : taille de la mère (en pouces)
- *weight* : poids de la mère (en livres)
- *smoke* : 1 si la mère est fumeuse, 0 sinon.

Commencez par charger les données du ficher *bebe.csv* dans **Python**. Supprimez la variable X qui est inintéressante pour notre évaluation.

```
> import pandas as pd
> import numpy as np
> import matplotlib.pyplot as plt
> import seaborn as sns
> from scipy import stats

> Data = pd.read_csv('bebe.csv')
> df = pd.DataFrame(data = Data)
> print(df.describe())
                 X          bwt    gestation       parity
                 age  \
count  1174.000000  1174.000000  1174.000000  1174.000000
   1174.000000
mean    587.500000  3386.702747   279.101363     0.262351
   27.228279
std     339.048915   519.608671    16.010305     0.440100
   5.817839
min       1.000000  1559.222500   148.000000     0.000000
   15.000000
25%     294.250000  3061.746000   272.000000     0.000000
   23.000000
50%     587.500000  3401.940000   280.000000     0.000000
   26.000000
75%     880.750000  3713.784500   288.000000     1.000000
   31.000000
max    1174.000000  4989.512000   353.000000     1.000000
   45.000000

            height       weight        smoke
count  1174.000000  1174.000000  1174.000000
mean    162.685486    58.276913     0.390971
std       6.416298     9.404905     0.488176
min     134.620000    39.462504     0.000000
25%     157.480000    51.822886     0.000000
```

```
50%      162.560000    56.699000      0.000000
75%      167.640000    63.049288      1.000000
max      182.880000   113.398000      1.000000

> mydf = df.drop('X', axis=1)
> print(mydf.head())


         bwt  gestation  parity  age  height     weight  smoke
0  3401.9400        284       0   27  157.48  45.359200      0
1  3203.4935        282       0   33  162.56  61.234920      0
2  3628.7360        279       0   28  162.56  52.163080      1
3  3061.7460        282       0   23  170.18  56.699000      1
4  3855.5320        286       0   25  157.48  42.184056      0
```

À vous !

a) Commentez les valeurs médianes pour les variables quantitatives et les modalités les plus représentées pour les variables qualitatives.

b) Y-a-t-il des valeurs manquantes ? Pourquoi cela peut-il être un problème pour l'entraînement du modèle ?

1.2 Observation des variables et création des dataframe

Nous devons vérifier la répartition des données et les relations entre chacune des variables explicatives et la variable de réponse. Commencez par exécuter la fonction `pairplot` de la librairie *seaborn* :

```
#create pairs plot for all numeric variables
> sns.pairplot(mydf)
```

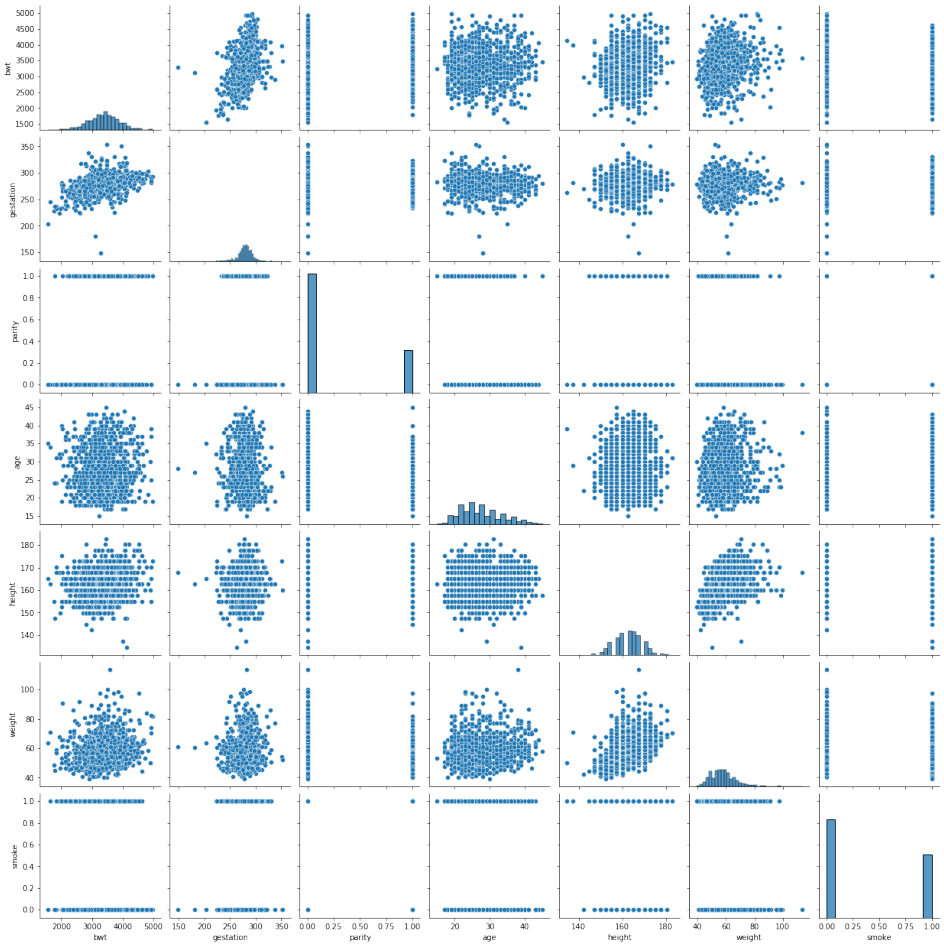


FIGURE 1 – Résultat de la fonction `pairplot` sur le dataset *bebe.csv*

.

À vous !

a) Identifiez les forte corrélations entre les variables.

b) Identifiez les faibles corrélations entre les variables.

c) Y-a-t-il plus de mère fumeuses ou non fumeuse ?

d) Pour construire un bon modèle, nous devons d'abord diviser notre ensemble de données en un ensemble d'entraînement et un ensemble de test. Construire les dataframe X_test, X_test, Y_train, Y_test à l'aide de la fonction `train_test_split` de *sklearn.model_selection* en utilisant 70% de l'échantillon pour des données de train et les 30% restants pour les données de test (on fixera la graine à 42). Y_train, Y_test seront associés à la variable à prédire et X_test, X_test aux variables explicatives.

1.3 *DecisionTreeRegressor* du Package *scikit-learn*

Nous utiliserons l'implémentation de sklearn d'un d'arbre de régression et, pour apprendre à l'utiliser, voici les éléments suivants docs ou vous pouvez mettre un point d'interrogation avant un appel l'objet. L'ajout d'un ? à une méthode, une variable ou une fonction affichera la doc définie de cette méthode.
Cette fonction *DecisionTreeRegressor* génère un arbre par partitionnement récursif binaire en utilisant la variable qu'on souhaite prédire (mise dans une formule spécifiée en entrée) et en fractionnement les variables explicatives. La variable à prédire doit être continue pour un arbre de **régression** et binaire pour un arbre de **classification**. La variable à prédire est divisée en $X < a$ et $X > a$, de façon itérative et en faisant varier la valeur de a. La scission a est la valeur qui maximise la réduction de la variance (pour un abre de régression et l' "impureté" pour un arbre de classification . Le fractionnement itératif est poursuivi jusqu'à ce que les nœuds terminaux soient trop petits ou trop peu différentiables pour être divisés. Appliquez la fonction `DecisionTreeRegressor` sur votre dataframe pour prédire la variable `btw` à partir de toutes les variables.

```
> from sklearn.tree import DecisionTreeRegressor
> ?DecisionTreeRegressor
Init signature:
DecisionTreeRegressor(
    *,
    criterion='squared_error',
    splitter='best',
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    min_weight_fraction_leaf=0.0,
    max_features=None,
    random_state=None,
    max_leaf_nodes=None,
    min_impurity_decrease=0.0,
    ccp_alpha=0.0,
)
Docstring:
A decision tree regressor.
...
```

Nous allons construire un arbre assez simple pour le début. Il faut dans un premier temps créer l'objet avec ses paramètres et dans un second temps l'appliquer à un dataframe.

```
# Fit regression model
> regr_1 = DecisionTreeRegressor(max_depth=2)
> regr_1.fit(X_train, Y_train)
```

Nous allons utiliser la librairie *matplotlib* et *tree* de *sklearn* pour afficher l'arbre obtenu. Il faut spécifier les informations/dimensions que l'on souhaite obtenir pour faciliter la lecture.

```
> import matplotlib
> matplotlib.use('nbagg')
```

```
> import matplotlib.pyplot as plt

>  from sklearn import tree
>  dt_feature_names =  list(X_train.columns)
# plot tree
>  plt.figure(figsize=(30,10))  # set plot size (denoted in inches)
>  tree.plot_tree(regr_1, fontsize=10, filled=True ,  feature_names
   =dt_feature_names)
>  plt.show()
```

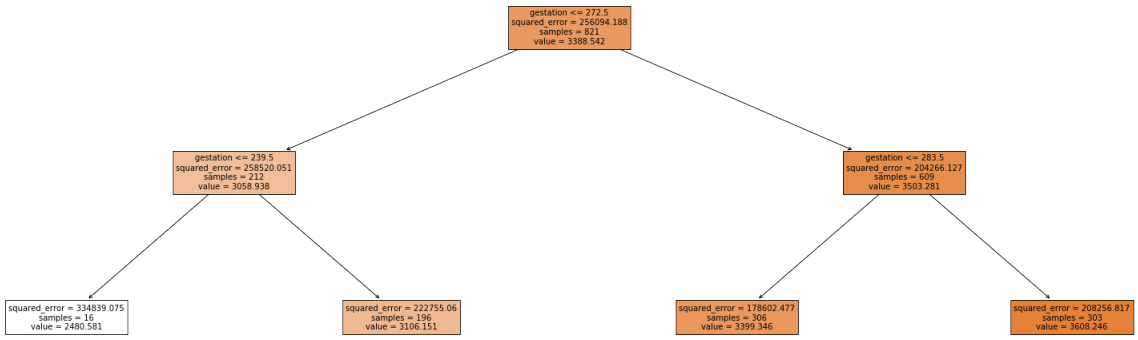


FIGURE 2 – Résultat de *DecisionTreeRegressor* (max_depth=2) de sur le dataset *bebe.csv*

.

Vous remarquerez dans l'arbre de la figure 1.3 que vous avez exclusivement des variables quantitative. Si vous souhaitez intégrer des variables binaires, il faudra faire une transformation, ce que nous verrons plus tard.
Nous allons maintenant tester notre modèle sur des données qu'il n'a pas vue. Pour cela, on lui demande de prédire pour le jeu de test le poids en ayant uniquement accès aux autres variables (explicatives), c'est à dire `gestation`, `parity`, `age`, `height` et `smoke`. Un premier moyen d'évaluation et d'observer le R^2, la *mean square error*, et la *root mean square error*. Une seconde évaluation, consiste à observer visuellement les données prédites par rapport aux données réelles.

```
> y_1 = regr_1.predict(X_test)
> from sklearn.metrics import mean_squared_error
> mse = mean_squared_error(Y_test, y_1)
> score =  r2_score(Y_test, y_1)

> print("R-squared:", score)
> print("MSE: ", mse)
> print("RMSE: ", mse**(1/2.0))
> plt.scatter(Y_test, y_1, )
> plt.title("Y r el vs Y pr dit")
> plt.xlabel("Y r el")
```

```
> plt.ylabel("Y pr dit")
> plt.show()
```

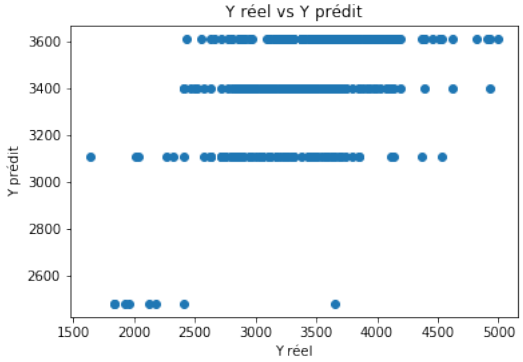


FIGURE 3 – Résultat de *DecisionTreeRegressor* (max_depth=2) sur le dataset *bebe.csv*

.

À vous !

a) Commentez l'arbre obtenu (description des groupes). Quelle est/sont les variables de *split* les plus discriminantes ?

b) Donnez la formule pour calculer manuellement le R^2 et la MSE Retrouvez les résultats obtenus avec *sklearn*.

c) Commentez le $R2$ obtenu. Comparez le avec le R^2 issue des données d'entraînement. Est-on en *underfit* ou *overfit* ?

d) Commentez le graphe 1.3. Commnent se comporte le modèle.

1.4 Parameter tuning

Pour réduire la variance, nous contraignons le modèle en utilisant certains des paramètres fournis, par exemple : - `criterion` (Fonction de coût utilisée pour mesurer la pureté d'un fractionnement) - `max_depth` taille maximale de l'arbre - `min_samples_split` Echantillons minimums pour chaque nœud scindé - `min_samples_leaf` Echantillons minimums pour chaque noeud terminal - `max_leaf_nodes` Nombre maximal de nœuds terminaux.

Vous pouvez également trouver plus d'informations sur ce blog pour un rappel des paramètres de l'arbre de décision.
Dans un premier temps, nous allons expérimenter avec le paramètre `max_depth` uniquement. Par le code suivant vous permet par exemple de comparer y_1 et y_2 (prédictions issues sous deux paramétrages différents) selon la variable explicative `gestation`.

```
> X_test_temp = X_test.loc[:, X_test.columns == "gestation"]
```

```
# Plot the results
> plt.figure()
> plt.scatter(X_test_temp, Y_test, s=20, edgecolor="black", c="
   darkorange", label="data")
> plt.plot(X_test_temp, y_1,'o', color="cornflowerblue", label="
   max_depth=2")
> plt.plot(X_test_temp, y_2, 'o',color="yellowgreen", label="
   max_depth=3")
> plt.xlabel("gestation")
> plt.ylabel("target")
> plt.title("Decision Tree Regression")
> plt.legend()
> plt.show()
```

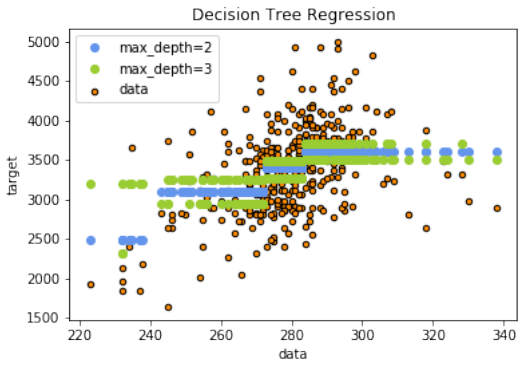


FIGURE 4 – Comparaison de *DecisionTreeRegressor* max_depth=2 et max_depth=3 selon la *gestation* sur le dataset *bebe.csv*

.

À vous !

a) Construisez un arbre avec une profondeur maximale de 3.
b) Interprétez l'arbre obtenu. Quelle autre variable a fait son apparition ?
c) Quelles sont les variables qui influencent le poids d'un nourrisson ?
d) Comparez les résultats avec le premier modèle de sort d'obtenir le graphique 1.4.
e) Comparez le R^2 pour les deux modèles
f) Comparez la MSE et la $RMSE$ pour les deux modèles
g) Quel est le meilleur modèle ?
h) Pensez-vous que l'abre de décision est une méthode adaptée pour ce jeu de données. Si ce n'est pas le cas, quel autre modèle pourrait être utiliser

2 Les arbres de classification

Lorsque les variables explicatives sont de qualitatives, vous utiliserez un arbre de classification. Avec des variables qualitatives, le nœud séparera les modalités de cette

variables en deux ensembles disjoints. Si nous avons n modalités possible, pour une variable qualitative X, il y aura alors $2^{n-1} - 1$ conditions de séparations possible sur cette variable. L'algorithme utilisé ici ($CART$) va tester toutes les possibilités et chercher à maximiser l'inertie intra-classe et minimiser l'inertie interclasse. Si notre variable cible est-elle aussi qualitative, les classes afficheront probabilité d'avoir une) ou plusieurs modalités (selon si votre variable à prédire est binaire ou multi-modale). Comme pour les arbres de régression, l'algorithme sélectionne les variables qui ont le meilleur critère de séparation, et s'arrête lorsque la séparation n'est plus utile.

2.1 Chargement des données

Dans cet exemple nous allons observer les passagers du Titanic et identifier les variables qui joué un rôle dans leurs probabilités de survie. Vous pouvez télécharger les données sur `https://vincentarelbundock.github.io/Rdatasets/datasets.html`.
Nous avons 1316 passagers avec les informations suivantes :

- X qui est l'ID du passager
- class : la classe du passager (1ier,2ieme,3ieme)
- age : deux catégories d'age : adulte ou enfant
- sex : si le passager est un homme ou une femme
- survived : si le passager a survécu (yes) ou pas (no)

FIGURE 5 – Un modèle prédictif ?

source :`http://kingsunraj.blogspot.fr`

```
> import numpy as np
> import pandas as pd
> from sklearn.model_selection import train_test_split
> myData = pd.read_csv('titanic.csv')
> print(myData.head())
   id      class     age  sex survived
0   1  1st class  adults  man      yes
1   2  1st class  adults  man      yes
2   3  1st class  adults  man      yes
3   4  1st class  adults  man      yes
4   5  1st class  adults  man      yes
```

À vous !

a) Commentez les données.

b) Supprimez la variable *id* qui n'est pas une donnée exploitable.

c) Quelles sont les variables binaires et quelles sont les variables à plus de deux modalités ? Utilisez la fonction `describe()` pour répondre à cette question.

2.2 Transformation des données et création des dataframes

La fonction `DecisionTreeClassifier` de *sklearn* ne permet pas de traiter les données catégorielles en tant qu'objet. Vous devez transformer ces variables en variables binaires (ne donc que des 0 et des 1). Par exemple une transformation de la variable `sex` aboutira à deux colonnes `sex_man` et `sex_women`. Si une personne est de sexe féminin, elle aura 1 à la variable `sex_women` et 0 pour `sex_man` (le même principe est appliqué pour les variables catégorielles multi-modale). Pour effectuez cette transformation, appliquez le code suivant

```
> import panda as pd
> myData_bin = pd.get_dummies(myData, columns=['class','age','sex
    '])
```

Une fois ces données transformées, nous allons séparer les données en deux dataframe respectivement pour les covariables et la variable à prédire et définir nos dataframes de train/test.

```
> X, y = myData_bin.loc[:, myData_bin.columns != 'survived'] ,
    myData_bin['survived']
print(X.head())
> X_train, X_test, y_train, y_test =  train_test_split(X, y,
    train_size=0.7, random_state=0)
```

Passons maintenant à la création du modèle !

2.3 Création de l'arbre

Des arbres de classification et de régression peuvent être générés à travers le package *skitlearn*. La fonction `DecisionTreeClassifier` utilise un critère choisi en paramètre (Entropie ou de indece de Gini par exemple) pour choisir la meilleur séparation de ces nœud. L'entropie de Shannon, due à Claude Shannon, est une fonction mathématique qui, intuitivement, correspond à la quantité d'une variable d'intérêt contenue dans un ensemble présentant d'autres d'informations.
Le coefficient de Gini due à Corrado Gini, est équivalent à l'écart moyen relatif (l'écart moyen divisé par la moyenne pour le mettre à l'échelle) : il s'agit donc bien d'une mesure de dispersion de valeurs numériques [2].

2. 0 signifie l'égalité parfaite et 1 signifie une inégalité parfaite

2.4 Entropie de Shannon

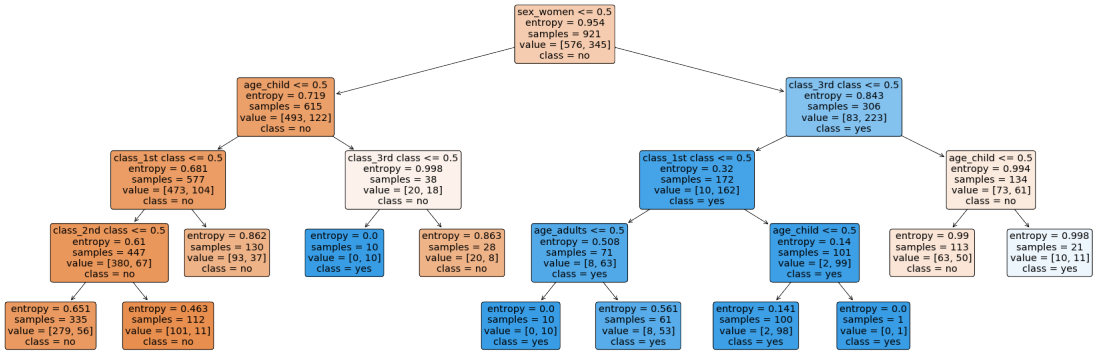
L'entropie mesure de "l'incertitude" contenue dans un jeu de données.
Soit un problème de classification avec K classes possibles avec des proportions p_i pour $i = 1$ à K. On définit l'entropie (dans le cas de deux classes possiblez), telle qu'une entropie égale à 0 indique une concentration unique d'une seule classe. *A contrario*, une entropie de 1 indique une répartition équilibrée. Dans le cadre de la construction de l'arbre, la variable de *split* est celle qui minimise l'entropie deux feuilles obtenues (pour plus d'informations, consultez le TD2). Le code pour obtenir un arbre basé sur l'entropie est le suivant

```
from sklearn import tree

clf = tree.DecisionTreeClassifier(criterion='entropy')
clf.fit(X_train, y_train)
cn=[ 'no','yes']
dt_feature_names =  list(X.columns)

# plot tree
plt.figure(figsize=(30,10))  # set plot size (denoted in inches)
plot_tree_cor  = tree.plot_tree(clf,class_names=cn, filled=True,
   feature_names=dt_feature_names, )
plt.show()
```

FIGURE 6 – Arbre de classification basé sur l'entropie



À vous !

a) Pour quelle raison certain noeuds terminaux peuvent avoir une entropie nulle ?

b) Quel noeud terminal a le plus grand échantillon ? Quelle est la prédiction associée ?

c) Quelle sera la prédiction (classe et probabilité) pour un petit garçon de seconde classe ?

d) La 2ème classe a-t-elle privilégié les femmes et les enfants d'abord ? La 1ère ? La 3ème ?

e) Donnez la formule mathématique de calcule de l'entropie de Shannon.

2.5 Coefficient de Gini

Pour conserver la distribution des classes, on utilise l'indice de Gini. Plus les classes sont uniformément distribuées dans un nœud, plus l'indice de Gini est élevé ; plus un nœud est pur, plus son indice de Gini est bas. Dans le cas de deux classes, l'indice de Gini va de 0 (nœud pur) à 0,5 (mélange maximal). Intuitivement, l'indice de Gini mesure la probabilité que deux individus choisis aléatoirement (avec remise) dans un nœud, appartiennent à deux classes différentes.
Á chaque séparation, l'arbre va se construire en cherchant à donner la plus grande pureté à ses nœuds fils, et donc baisser l'indice de Gini.

À vous !

a) Construisez un arbre basé sur l'indice de Gini et comparez votre modèle au précédent, en regardant notamment les quatre question précédentes.

b) Quels sont les autres paramètres possibles de la fonction `DecisionTreeClassifier` ?

c) Donnez la formule mathématique de calcule du coefficient de Gini

d) Que reste-il à faire pour évaluer la qualité de notre modèle ?

3 Évaluation du modèle

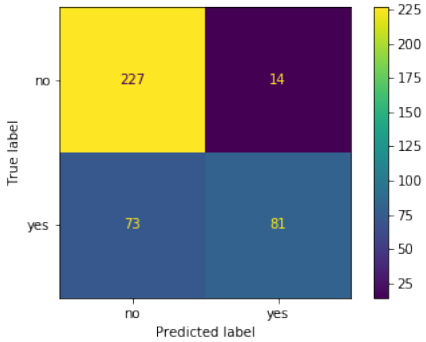
Nous allons regarder différentes métriques de qualité pour notre modèle. Vous connaissez déjà la Hinge Loss et Log loss. Vous pouvez également utilisez une matrice de confusion. Cette matrice mesure la qualité d'un système de classification : chaque ligne correspond à une classe réelle, chaque colonne correspond à une classe estimée. On peut ainsi voir la correspondance entre le nombre d'éléments de la classe réelle qui ont été bien/mal classé. Un des intérêts de la matrice de confusion est qu'elle montre rapidement si un système de classification parvient à classifier correctement.

```
#Version simple
> from sklearn.metrics import confusion_matrix
> cm = confusion_matrix(y_test, clf.predict(X_test))
> print('Confusion matrix\n\n', cm)
Confusion matrix

 [[227  14]
 [ 73  81]]

#Version plus jolie
>  from sklearn.metrics import plot_confusion_matrix
> plot_confusion_matrix(clf, X_test, y_test)
> plt.show()
```

FIGURE 7 – Matrice de confusion pour modèle basé sur l'entropie appliqué au test set de *Titanic.csv*



À vous !

a) Quel est le taux de bonne classification ?

b) Combien de 'survivants' ont été classé comme 'morts' ?

c) Quelle erreur commet majoritairement le modèle ?

d) Donnez la formule mathématique de la Hinge Loss et la Log Loss.

e) À l'aide de la librairie *sklearn.metrics*, calulez la Hinge Loss et la Log Loss des deux modèles (Entropie et Gini).

f) Comparez la matrice de confusion des deux modèles.

4 Pour aller plus loin

`https://towardsdatascience.com/build-better-decision-trees-with-pruning-8f467e73b1`
`https://www.youtube.com/watch?v=D0efHEJsfHo`,
`https://towardsdatascience.com/an-exhaustive-guide-to-classification-using-decision`